

大模型思维链原理

从CoT到ToT，解锁AI的深度推理能力

本次研讨的探索路径

1. 问题的提出：为何大模型需要“思考过程”？

- 标准提示（Standard Prompting）的局限性

2. 思维链（CoT）：引导模型“思考”的第一步

- 核心思想与性能提升
- 进阶：自洽性思维链（CoT-SC）

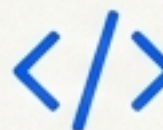
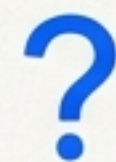
3. 思维树（ToT）：从线性推理到策略探索

- 核心机制：生成、评估、搜索

4. 实战拆解：“24点游戏”中的ToT实现

- 代码级深入解析

5. 总结与展望：思维链技术的核心启示



强大的语言模型为何在简单推理上犯错？

标准提示 (Standard Prompting)

Q: Roger有5个网球。他又买了2罐网球。每罐有3个网球。他现在有多少个网球？

A: 答案是11。



模型直接给出了一个看似合理但错误的答案，因为它没有分解问题并按步骤计算。

问题的核心



大模型在处理需要多步推理、规划或复杂计算的任务时，若直接输出答案，很容易出错。它们需要一个框架来引导其“思考”过程。

- 🚫 缺乏中间步骤的可解释性。
- 📉 一步错，步步错，无法纠正。

解决方案：引导模型展示“思维链”（Chain-of-Thought）

引导LLM将复杂问题分解出中间步骤，提供更丰富的可解释性。这通常通过小样本(few-shot)方式引导。

标准提示（失败案例）

Q: 食堂有23个苹果。他们用20个做午餐，又买了6个。现在食堂有多少苹果？

A: 答案是27。



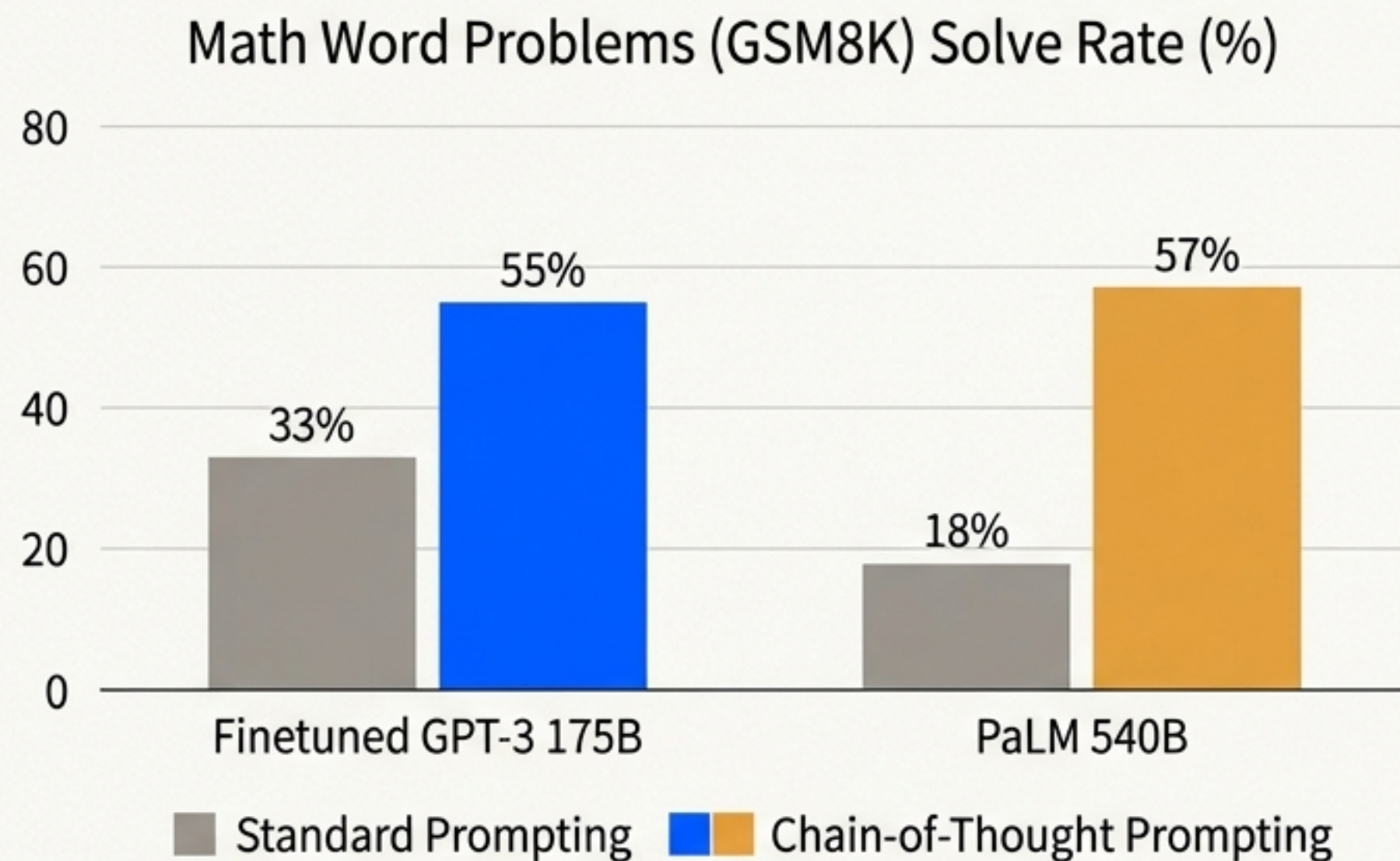
思维链提示（成功案例）

Q: 食堂有23个苹果。他们用20个做午餐，又买了6个。现在食堂有多少苹果？

A: 食堂原来有23个苹果。他们用掉了20个，所以还剩 $23 - 20 = 3$ 个。他们又买了6个，所以现在有 $3 + 6 = 9$ 个。
答案是9。



数据证明：思维链显著提升数学推理能力



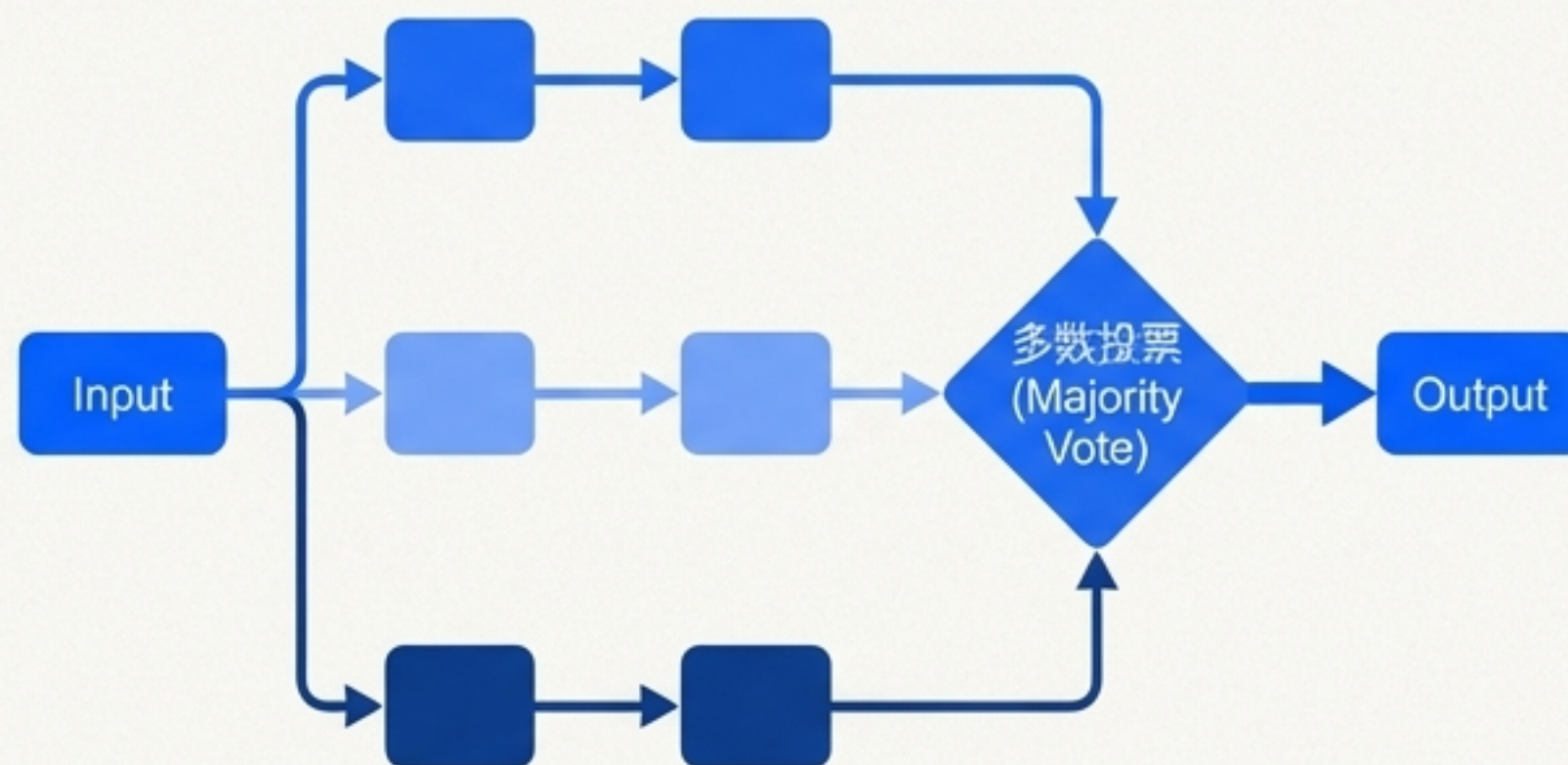
- **基线：**PaLM 540B (Standard Prompting) 的解决率仅为 **18%**。
- **巨大飞跃：**采用思维链提示 (Chain-of-Thought Prompting) 后，解决率飙升至 **57%**。
- **微调模型：**经过微调的GPT-3 175B模型，解决率也从33%提升到 **55%**。

关键结论：CoT不仅仅是让输出更可读，它从根本上提升了模型在复杂推理任务上的准确性。

超越单路径：自治性思维链（CoT-SC）提升鲁棒性

Core Idea

- **问题：**单一的思维链仍然可能出错。如果第一步就错了，整个推理就会失败。
- **解决方案：**生成多条不同的推理路径（思维链），然后通过“多数投票”（Majority Vote）的方式选出最可靠的最终答案。



示例

→ 路径1: " $9-3=6$... 答案是\$18。"

→ 路径2: " $9/3=3$... 答案是\$24。"

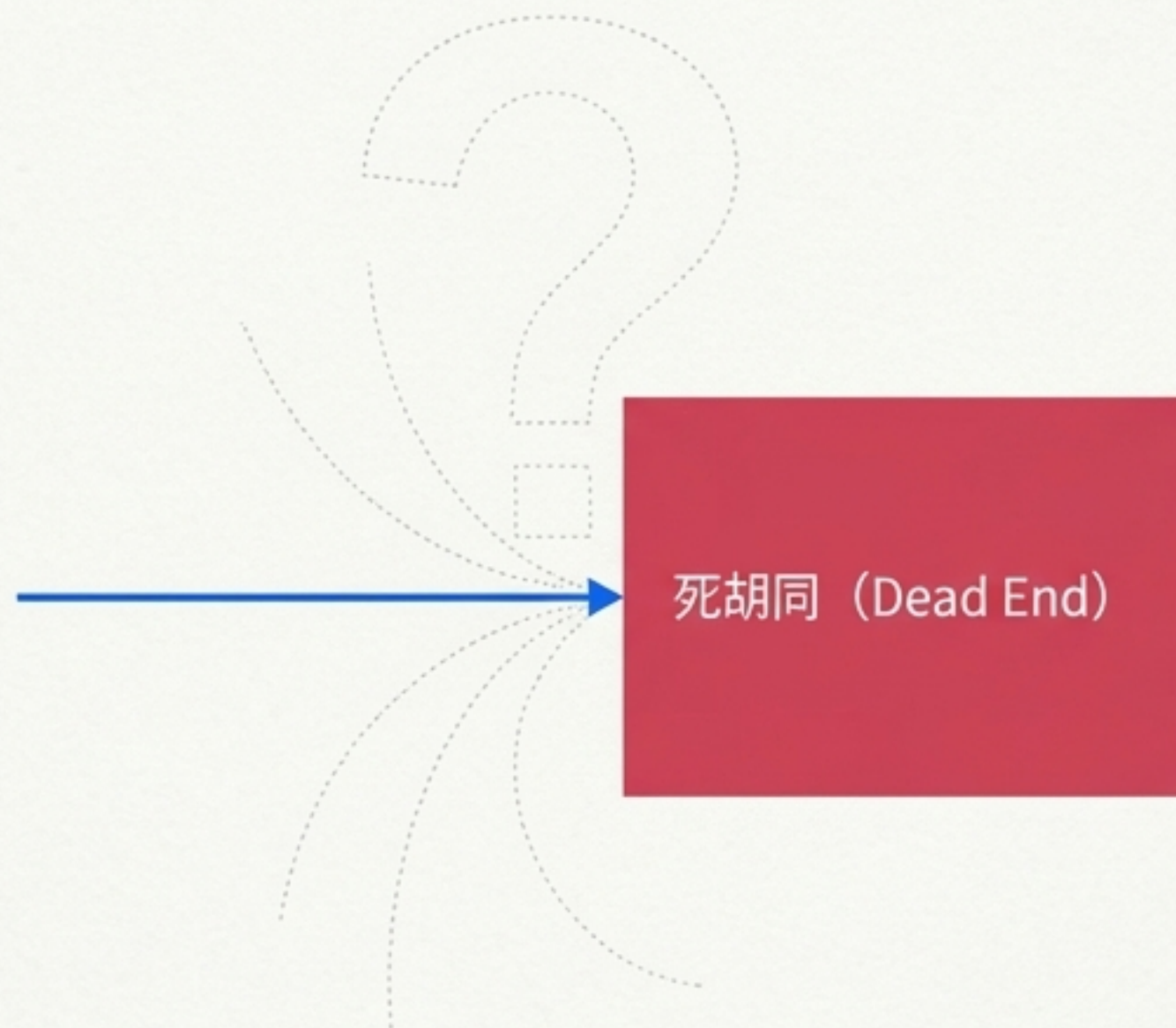
→ 路径3: " $3 \times 3=9$... 答案是\$18。"

最终答案：通过投票，选择\$18。

思维链的局限：当线性探索不足以解决问题

Core Challenges

- CoT 和 CoT-SC 本质上是“贪心解码”（Greedy Decode）的延伸。它们在每一步都选择看似最好的选项，但无法进行全局规划或在发现错误时回溯。
- 对于需要广泛探索和战略性选择的任务（如游戏、规划），线性思维链的局限性就显现出来了。



Provocative Questions

我们能否让模型在推理过程中主动探索不同的可能性？

模型能否自我评估中间步骤的“好坏”？

当一条路走不通时，模型能否“回溯”并尝试其他路径？

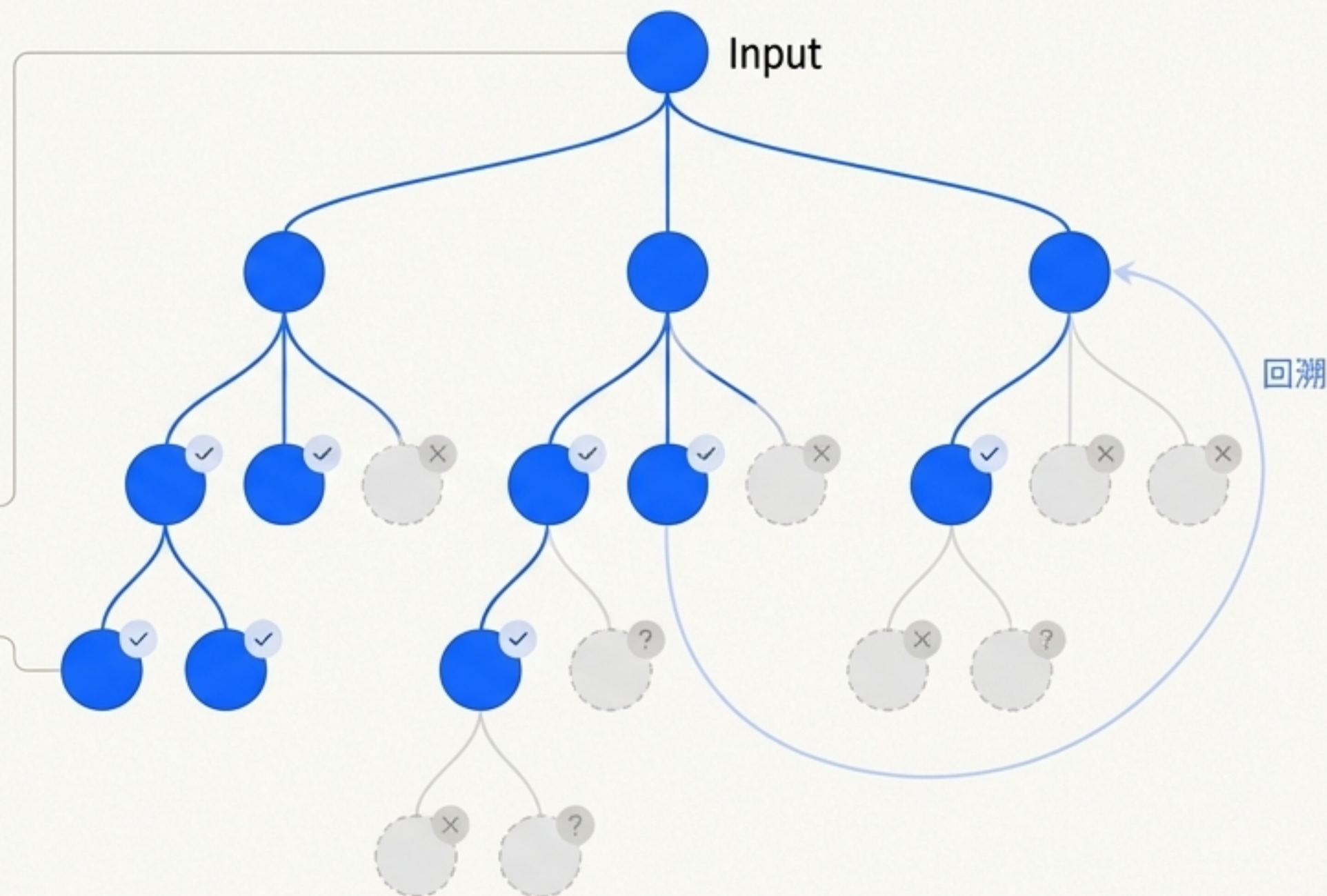
思维的进化：思维树（Tree of Thoughts, ToT）

Core Idea

ToT将大模型的推理过程从一个“链条”泛化为一个“树”。它允许模型在每一步探索多个分支，并使用启发式搜索来指导决策。

ToT's Core Steps

1. **想法分解**（Thought Decomposition）：将问题分解为多个思考步骤。
2. **想法生成**（Thought Generation）：在每个步骤中，生成多个可能的后续想法（分支）。
3. **状态评估**（State Evaluation）：对每个生成的想法进行评估，判断其“价值”或“前景”。
4. **搜索算法**（Search Algorithm）：使用搜索策略（如广度优先BFS或深度优先DFS）来系统地探索这棵思维树，并可以进行回溯。



实战案例：用思维树挑战“24点游戏”

任务描述

- 目标：给定4个数字，使用加、减、乘、除运算，使得最终结果为24。
- 挑战：这是一个典型的需要探索和规划的问题。简单的线性推理很容易陷入死胡同。例如，给定 `4 4 6 8`，第一步 `4+4=8` 可能是错误的开端。



为何ToT是理想选择

- 每一步都有多种操作选择（例如，`4+4`，`4-6`，`4\*8` 等），天然形成一棵搜索树。
- 需要评估中间结果（例如，`8 6 8`）距离最终目标 `24` 的远近。
- 允许在一条路径看起来无望时，回溯并探索其他分支。

ToT代码拆解（1/3）：想法生成（Generation）

目标：在树的每个节点，生成所有可能的下一步操作。

实现方式：使用精心设计的 ``propose_prompt``，要求LLM基于当前数字状态，列出可能的计算步骤。

解释：这个Prompt向模型展示了一个例子，然后要求它为新的输入（如 ``{input}``）生成类似的“下一步”列表。这就是思维树如何“发芽”出新的分支。

game24.py

```
# 1-shot prompt
propose_prompt = """
Input: 2 8 8 14
Possible next steps:
2 + 8 = 10 (left: 8 10 14)
8 / 2 = 4 (left: 4 8 14)
14 + 2 = 16 (left: 8 8 16)
...
Input: {input}
"""
```


ToT代码拆解 (2/3) : 状态评估 (Evaluation)

目标：判断每个生成的“想法”（即中间计算结果）是否“有前途”。

实现方式：使用`value\_prompt`，要求LLM对当前状态给出一个评级（如“sure/likely/impossible”），判断它是否能最终导向24。

解释：模型通过这个Prompt学会了“直觉判断”。它会评估像`10 10 1`这样的中间状态，并给出“likely”的评价，因为它离24很近。这个评价将用于指导下一步的搜索。

game24.py

```
value_prompt = """
Evaluate if given numbers can
reach 24 (sure/likely/impossible)
10 14
> sure
11 12
> impossible
4 4 10
> likely
{input}
>
"""
```


ToT代码拆解（3/3）：搜索策略（Search）

目标：

结合“生成”和“评估”的结果，决定下一步探索哪个分支。

实现方式：

在核心的`solve`函数中，通过一个循环来迭代执行“生成 -> 评估 -> 选择”的过程。

解释：

这一步是ToT的大脑。它根据评估分数，决定是采用“贪心”策略（只走最好的路）还是“采样”策略（尝试一些看起来不错的路），从而高效地在巨大的可能性空间中寻找答案。

basetof/search/visualize.py

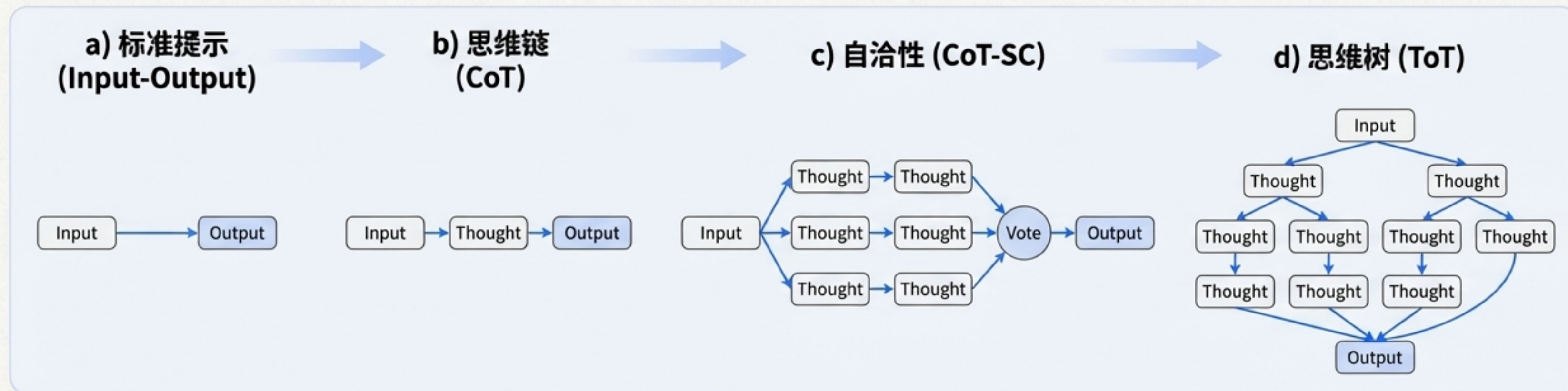
```
# Simplified logic of solve()
for step in range(task.steps):
    1 → # 1. Generation: Propose new thoughts
        new_ys = get_proposals(task, x, ys)

    2 → # 2. Evaluation: Get values for new thoughts
        values = get_values(task, x, new_ys)

    3 → # 3. Selection (Search Strategy)
        if args.method_select == 'sample':
            # Randomly select from promising candidates
            ids = np.random.choice(len(new_ys), ...)
        elif args.method_select == 'greedy':
            # Select the best candidates based on value
            ids = sorted(range(len(new_ys)), key=lambda i:
values[i], reverse=True)[:args.n_select_sample]

        # Prune the tree and select next states
        selected_ys = [new_ys[i] for i in ids]
```


推理框架的演进之路



核心演进

- 从 隐式推理 到 显式过程。
- 从 单一路径 到 多路探索。
- 从 被动执行 到 主动评估与回溯。

结论：这一演进过程的核心是让LLM的“思考”过程更加结构化、可控和强大。

核心启示与未来方向



1. 过程重于结果

对于复杂任务，引导模型生成推理过程（如CoT）是提升其能力的关键。



2. 探索优于贪心

允许模型探索多个可能性并从中选择（如ToT），能解决更复杂、更开放的问题。



3. 自评是智能的基石

让模型具备评估中间步骤好坏的能力，是实现真正规划和问题解决能力的核心。

展望

未来的大模型推理技术将继续沿着这条路发展，集成更复杂的搜索算法、更精准的评估函数，并可能将这些“思维”过程与外部工具（如代码执行器、计算器）深度结合，构建更强大的智能体。